Sesetti Gayathri
25NU1D7712

M.TECH MINI-PROJECT REPORT

Design and Analysis of Approximate Multipliers for Low-Power AI Applications

A hardware-efficient approximate 4-bit × 4-bit multiplier built on partial-product truncation, verified in Verilog and benchmarked for AI inference workloads.

Department of Electronics & Communication Engineering
Academic Year 2025-26

Introduction

A quick roadmap of what this presentation covers

01 **Abstract**

02 **Exact Multiplier Design**

03 **Approximate Multiplier Design**

04 **Synthesis Outputs (Vivado)**

05 **Simulation Results (GTKWave)**

06 **Error Analysis (Python Metrics)**

07 **Synthesis Results (Hardware Comparison)**

08 **AI Application Demo & Conclusion**

Abstract

Design and Analysis of Approximate Multipliers for Low-Power AI Applications

AI and deep learning applications are growing rapidly and need hardware that is fast, small, and uses less power. Multiplication is the most commonly used operation in AI systems and consumes a large share of hardware resources. Approximate computing allows hardware to make small, bounded errors — since AI applications remain accurate despite tiny errors, approximate multipliers help reduce power, area, and hardware complexity.

This project designs and analyzes a 4-bit × 4-bit approximate multiplier for low-power AI use. An exact multiplier is first designed in Verilog HDL using the standard partial-product method, always producing the correct answer and serving as the reference for comparison. An approximate multiplier is then designed by removing some smaller partial products and keeping only the important larger ones, making the circuit simpler and more power-efficient.

Both multipliers are written in Verilog HDL and tested using Icarus Verilog, with a testbench that checks all possible inputs and GTKWave used to view the simulation waveforms. Both designs are then synthesized on Xilinx Vivado to measure LUT usage, speed, and power consumption. Python is used to compare the outputs of both multipliers and calculate the error introduced by the approximate multiplier.

The main goal is to study the trade-off between hardware savings and accuracy, showing that approximate multipliers are a practical choice for low-power AI hardware.

Keywords: Approximate Computing, Approximate Multiplier, AI Applications, Partial Product Truncation, Verilog HDL, Low-Power VLSI.

Exact Multiplier — Partial Product Method

4-bit × 4-bit multiplication generates four partial products (pp0–pp3), shifted and summed to produce an 8-bit result — mathematically perfect for every input.

```
module exact_multiplier (
  input  [3:0] A, B,
  output [7:0] P
);
  wire [7:0] pp0 = B[0] ?
    {4'b0, A} : 8'b0;
  wire [7:0] pp1 = B[1] ?
    {3'b0, A, 1'b0} : 8'b0;
  wire [7:0] pp2 = B[2] ?
    {2'b0, A, 2'b0} : 8'b0;
  wire [7:0] pp3 = B[3] ?
    {1'b0, A, 3'b0} : 8'b0;

  assign P = pp0+pp1+pp2+pp3;
endmodule
```

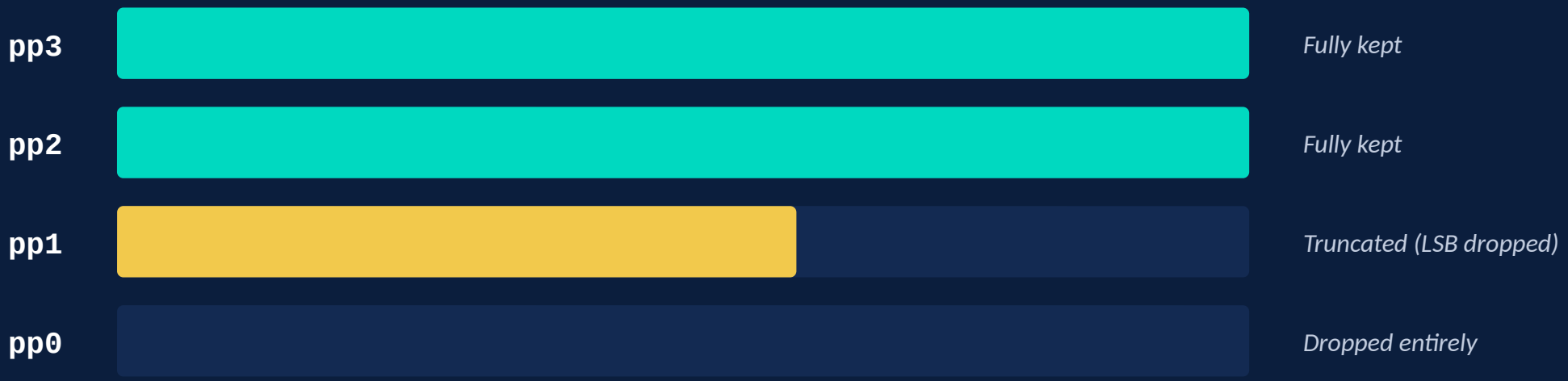
Worked Example: 10 × 12 = 120

Partial Product	Condition	Value
pp0	B[0]=0 → inactive	00000000
pp1	B[1]=0 → inactive	00000000
pp2	B[2]=1 → A<<2	00101000
pp3	B[3]=1 → A<<3	01010000
P = Σ ppk	**Sum**	**01111000 = 120**

Verified across all 256 possible 4-bit input combinations — zero error, 100% exhaustive coverage.

Approximate Multiplier — Partial-Product Truncation

Drop pp0 entirely • Truncate pp1 (LSB of A) • Keep pp2 & pp3 fully intact



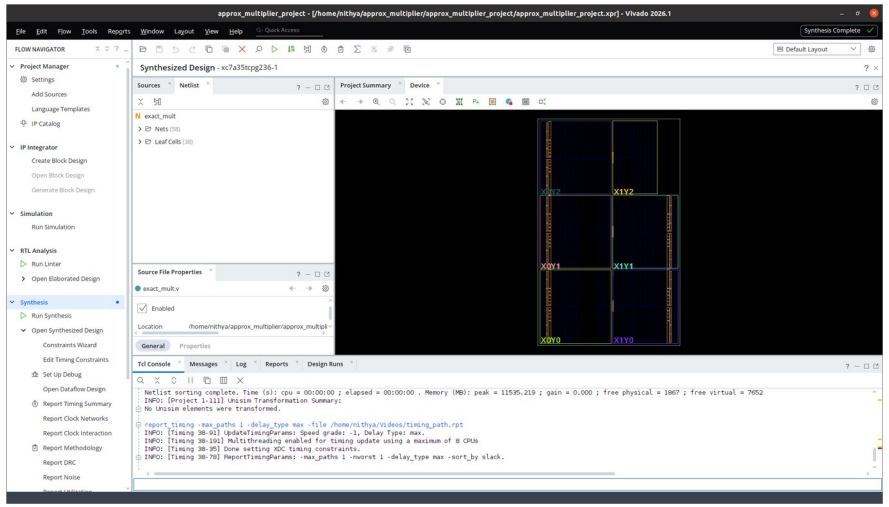

```
wire [7:0] pp3 = B[3]?{1'b0,A,3'b0}:8'b0;
wire [7:0] pp2 = B[2]?{2'b0,A,2'b0}:8'b0;
wire [7:0] pp1_approx = B[1]?
                {3'b0,A[3:1],2'b0}:8'b0;
// pp0 dropped
assign P = pp1_approx + pp2 + pp3;
```

One full adder level removed → shorter critical path, fewer LUTs, lower switching power.

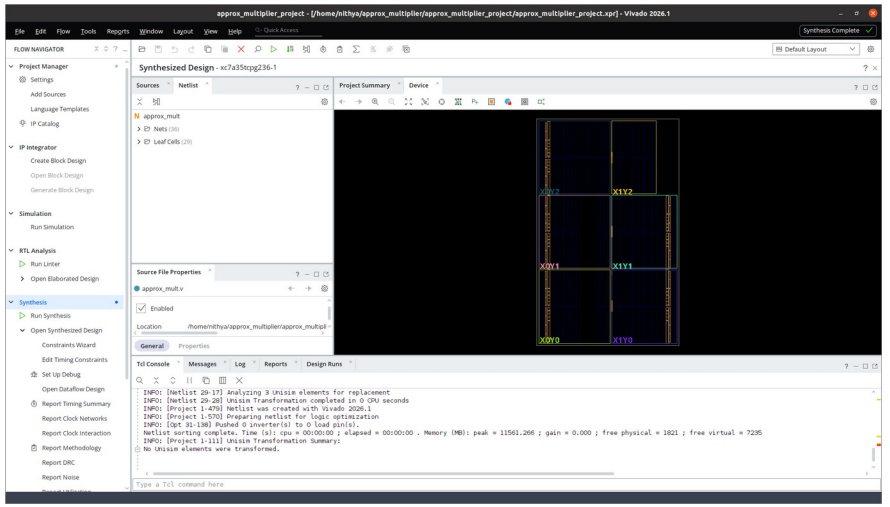
Synthesis Outputs

Vivado synthesized-design views for both multiplier variants (Xilinx Artix-7, xc7a35tcpg236-1)

Exact Multiplier — Synthesized



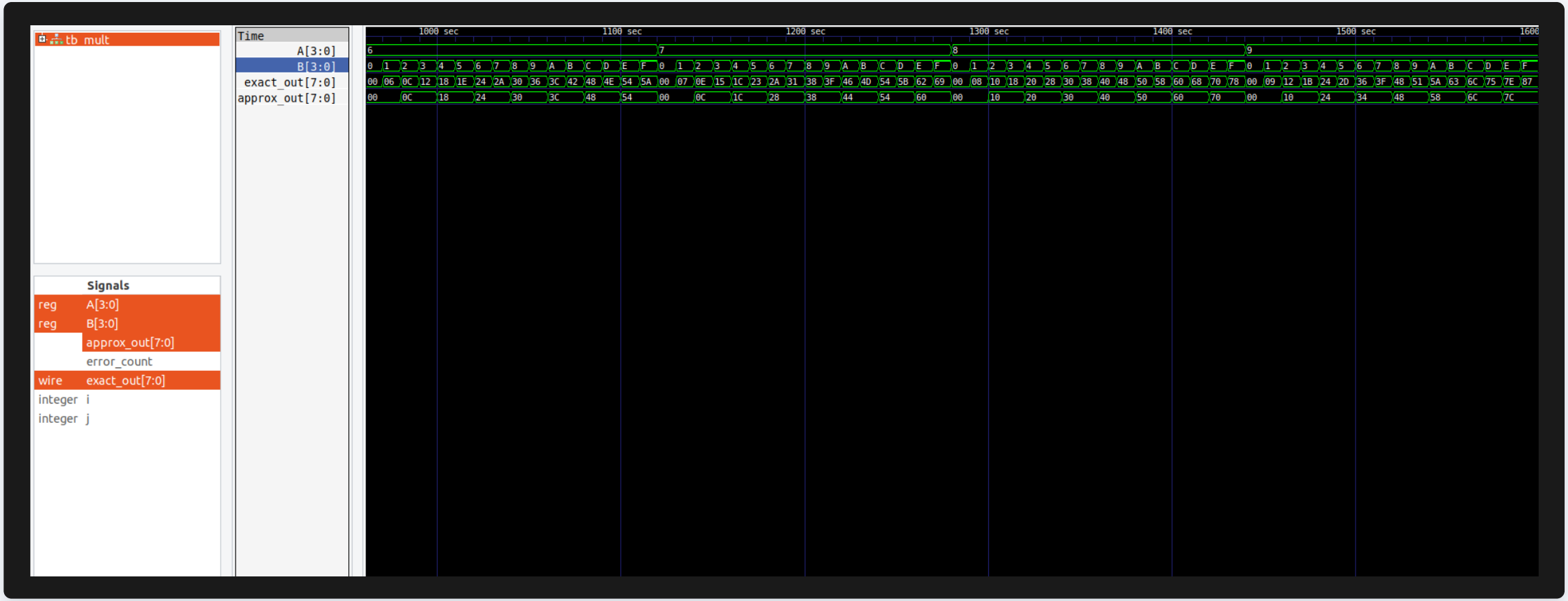
Approximate Multiplier — Synthesized



- **Leaf cells:** 38 (exact) vs 29 (approximate) — fewer synthesized cells confirm the reduced logic footprint.
- **Nets:** 58 (exact) vs 36 (approximate), reflecting the simplified partial-product interconnect.

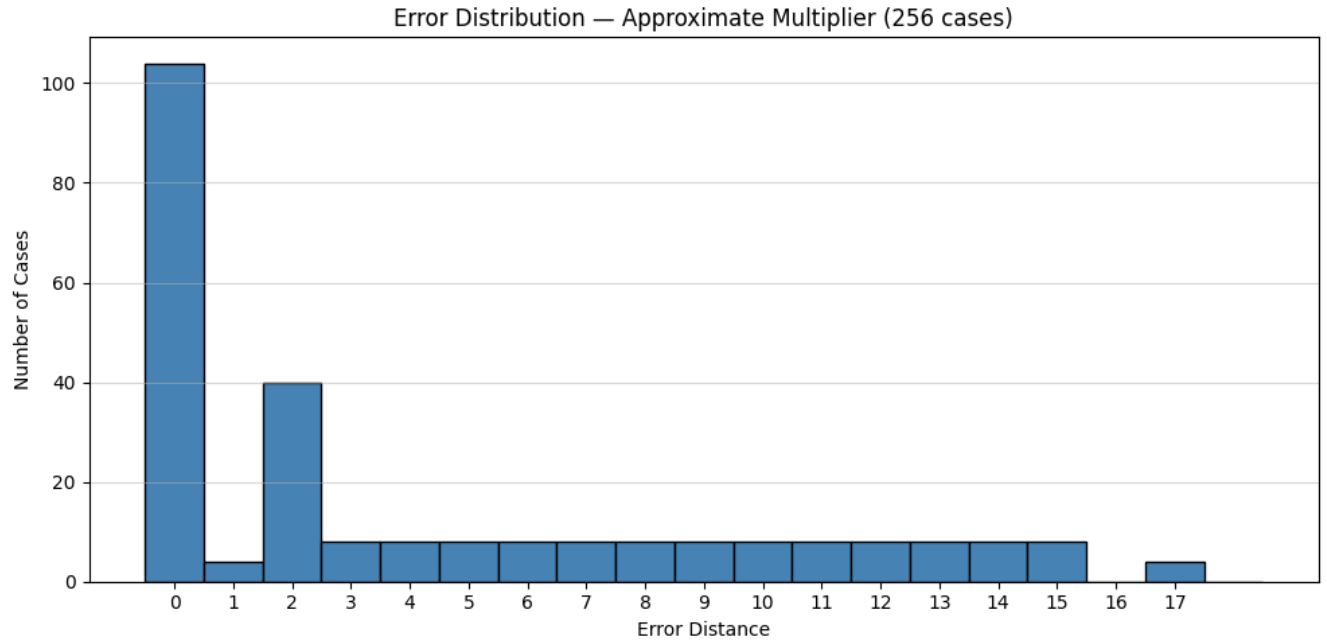
Simulation Results — GTKWave Verification

Common testbench applies all 256 input combinations to both designs simultaneously



Observation: Outputs match exactly for (3,5), (4,6), (6,6) — dropped bits don't contribute. Visible divergence appears for (1,8), (7,2), (15,15), (9,3), (12,11).

Error Analysis — Python Metrics



4.25

Mean Error Distance (MED)

59.38%

Error Rate (ER)

17

Max Error Distance

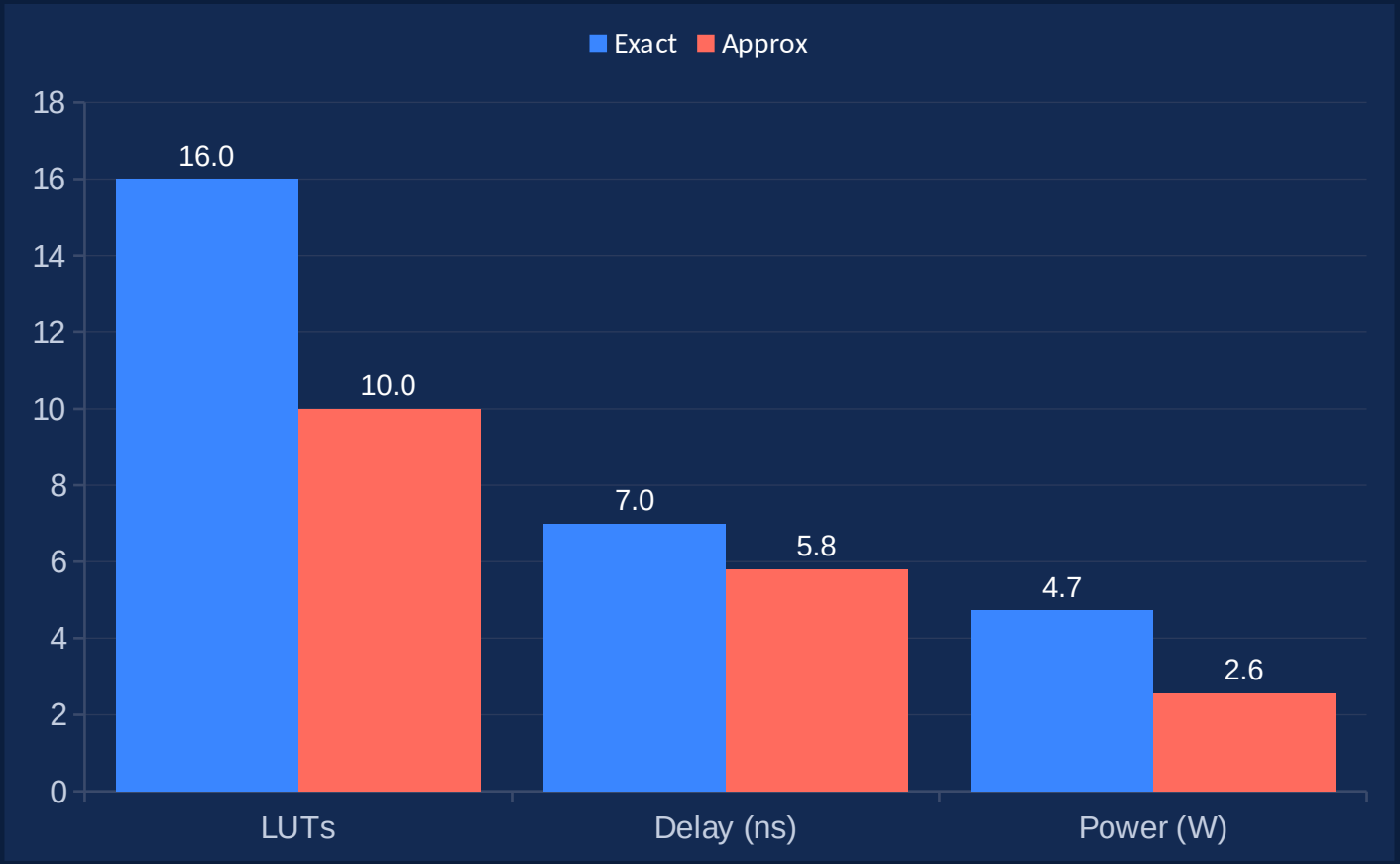
14.17%

Mean Relative Error (MRED)

Errors increase with operand magnitude but remain bounded (Max ED = 17) — predictable, AI-tolerable behavior.

Synthesis Results — Hardware Comparison

Vivado synthesis target: Xilinx Artix-7 (xc7a35tcpg236-1)



37.5% LUT Reduction

17.0% Delay Reduction

45.7% Power Reduction

AI Application Demo & Conclusion

Key Conclusions

- Exact multiplier verified with zero error across all 256 cases
- Approximate design cuts LUTs 37.5%, delay 17.0%, power 45.7%
- Error remains bounded (Max ED = 17) and data-dependent
- AI matrix-multiply demo confirms acceptable inference quality
- Suitable for low-power, edge-AI accelerator hardware

Bottom line: The trade-off — small, bounded error for real gains in area, delay, and power — makes this design well suited to applications where efficiency matters more than perfect precision.